

Documentation Technique – SupKrellM

Projet Python –Monitoring Système

Noa ALLIES & Nana Koita

Bachelor of science (Bs.c.1) / 2025 – 2026

Sommaire

- 1. Introduction - p3**
- 2. Description général du programme - p4**
- 3. Architecture des fichiers - p5**
- 4. Installation - p7**
 - 4.1. La page HTML - p24**
- 5. Méthodes de récupération des données - p25**
- 6. Mode Graphique (TKinter) - p30**
- 7. Les Paramètres du script - p32**
- 8. Gestion des Erreurs - p37**
- 9. Limitations Connues - p38**
- 10. Conclusion - p39**

1. Introduction

1.1 Le but du projet

1PRO1 – SupKrellM était de mettre en œuvre nos compétences en programmation système en développant une application en Python capable de générer une page web ou un interface graphique pour le rapport d'état d'un système Linux.

1.2 Contexte

Le programme devra collecter une série d'informations système pertinentes, puis les agréger et les présenter de manière claire et professionnelle dans un document HTML5 unique.

1.3 Objectif principal

Générer un rapport système en HTML statique.

1.4 Technologies imposées

Python 3 + bibliothèque standard uniquement.

1.5 Contraintes

pas de module os, extraction depuis /proc et /sys, pas de pip.

2. Description général du programme

SupKrellM est un ensemble de script Python qui permet de récupérer des informations précises d'un système Linux et de les présenter dans un rapport HTML propre et lisible.

Dans son fonctionnement global, le programme commence par collecter toutes les données système nécessaires, comme la mémoire, les disques, les processus ou encore les interfaces réseau. Une fois que ces informations sont récupérées, elles sont traitées pour devenir compréhensibles et adaptées à l'affichage (par exemple en convertissant des valeurs brutes en Go, en pourcentages ou en durées).

Le rapport HTML est basé sur un modèle avec 2 Templates :

(PAGE_MOEDELE) contient le header et le footer (ainsi que le bouton régénérer la page) et un iframe pour afficher la seconde page donc le second template.

(PAGE_DONNEES) contient lui, toute la structure importante du site ainsi que les informations à remplacer. On a donc placé des jetons du type %%MEM_TOTALE%% ou %%NOM_HOTE%%.

Le programme remplace automatiquement ces jetons par les valeurs réelles qu'il a calculées. Ce qui permet d'avoir un HTML propre, sans manipulation manuelle.

Le script génère deux fichiers HTML : un fichier principal qui sert d'interface, et un second fichier qui contient uniquement les données mises à jour. Ce choix permet d'actualiser les informations sans avoir à recharger toute la page (pour plus de confort).

Pendant toute son exécution, le programme met donc à jour les données automatiquement toutes les 2 secondes (selon la valeur définie par l'utilisateur (nous recommandons de ne pas modifier la valeur)). Cela permet d'avoir un suivi régulier de l'état du système sans interaction supplémentaire.

Schéma simple du fonctionnement :

Collecte données → Traitement → Remplissage du [PAGE_DONNEES] → Rapport HTML

3. Architecture des fichiers

Fichiers actuel :

1 : rapport_supkrellm.py (fichier qu'il faudra lancer)

2 : templates_rapport.py

3 : collecte_systeme.py

4 : utils.py

5 : Fiche_technique (fichier actuel)

Si il manque un fichier .py (1, 2, 3, 4), le programme ne fonctionnera pas !

Après la génération "rapport_supkrellm.py" :

1 : rapport_supkrellm.html (si pas modifié avec --page)

Ce fichier sera l'interface du site (c'est lui qu'il faudra ouvrir).

2 : donnees_systeme.html (si pas modifié avec --page)

Ce fichier sera les données actualisées (peut être ouvert mais n'est pas recommandé si nous voulons la structure entière du site (c'est le site qui sera dans l'iframe de l'interface)).

Après la génération "rapport_supkrellm.py --gui" :

1 : rapport.html

Ce fichier est créée seulement si l'on fait un Export HTML dans Tkinter

Fichier → Exporter HTML → rapport.html

Pourquoi deux fichiers séparés ? :

1. Pour éviter de recharger tout le rapport.
2. Pour garder les animations et la navigation.
3. Plus stable.

4. Installation

Prérequis :

1 – Linux

(Sur VMware Workstation Pro avec la distribution Ubuntu de préférence).

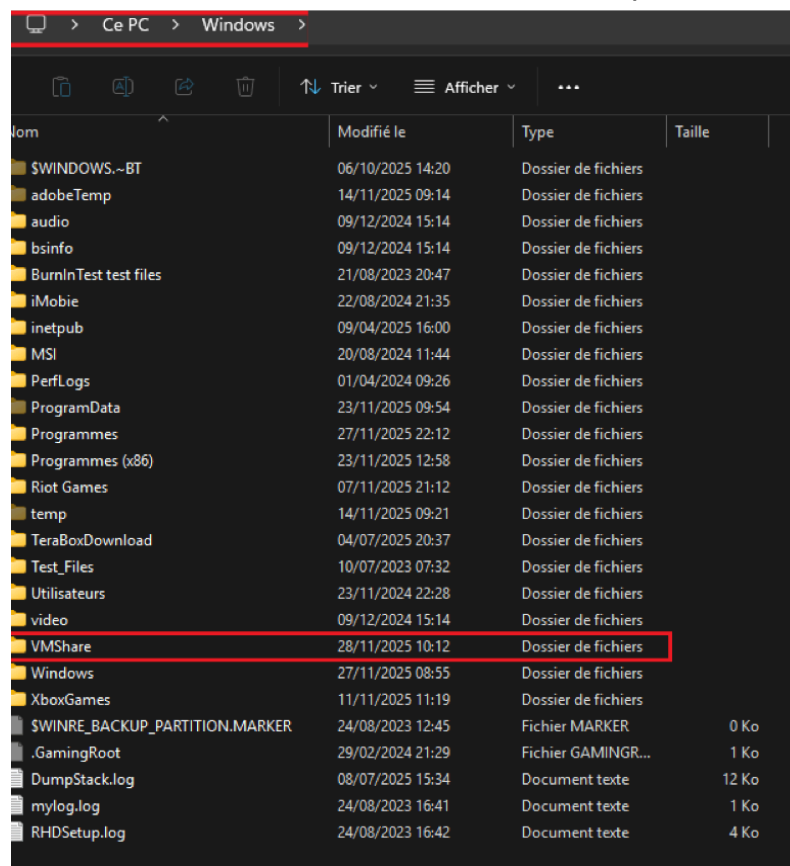
2 – Python 3.

3 – N'importe quel navigateur

(Chrome, Edge, Opera, Safari...).

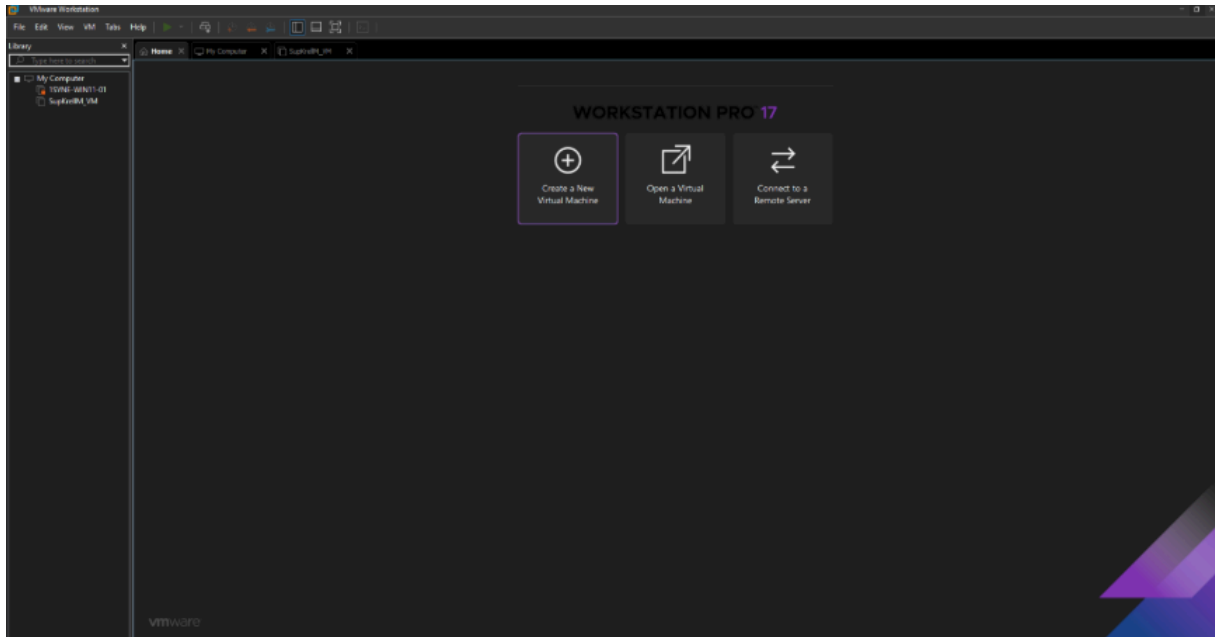
Comment procéder :

- 1) Vérifier que les 4 fichiers “.py” sont présents dans le même dossier que cette fiche technique.
- 2) Créer un Dossier “VMShare” dans votre disque C:/



- 3) Copier Coller tout les .py dans le dossier VMShare (sans les mettre dans un autre dossiers)
- 4) Installer VMware Workstation Pro si pas déjà fait.

- 5) Ouvrir VMware Workstation Pro.
- 6) Cliquer sur Create a New VM.



- 7) Vous devriez avoir une page comme celle si, merci de suivre les instructions :

7.1)



7.2) Ici, mettre le chemin du **.iso** de votre Linux qui possède un IG. peut être installé grâce à ce lien :

<https://releases.ubuntu.com/24.04.3/ubuntu-24.04.3-desktop-amd64.iso>

New Virtual Machine Wizard

Guest Operating System Installation
A virtual machine is like a physical computer; it needs an operating system. How will you install the guest operating system?

Install from:

☐ Installer disc:
No drives available

☒ Installer disc image file (iso):
NE\Virtualisation\ubuntu-24.04.3-desktop-amd64.iso Browse...

☒ Ubuntu 64-bit 24.04.3 detected.
This operating system will use Easy Install. [\(What's this?\)](#)

☐ I will install the operating system later.
The virtual machine will be created with a blank hard disk.

Help < Back Next > Cancel

7.3)

New Virtual Machine Wizard

Easy Install Information
This is used to install Ubuntu 64-bit.

Personalize Linux

Full name: SupKrellM

User name: User

Password: ●●●●●●

Confirm: ●●●●●●

Help < Back Next > Cancel

7.4)

New Virtual Machine Wizard ✕

Name the Virtual Machine
What name would you like to use for this virtual machine?

Virtual machine name:

Location:
 Browse...

The default location can be changed at Edit > Preferences.

< Back Next > Cancel

7.5) (Inutile de mettre plus)

New Virtual Machine Wizard ✕

Specify Disk Capacity
How large do you want this disk to be?

The virtual machine's hard disk is stored as one or more files on the host computer's physical disk. These file(s) start small and become larger as you add applications, files, and data to your virtual machine.

Maximum disk size (GB): ▲ ▼

Recommended size for Ubuntu 64-bit: 20 GB

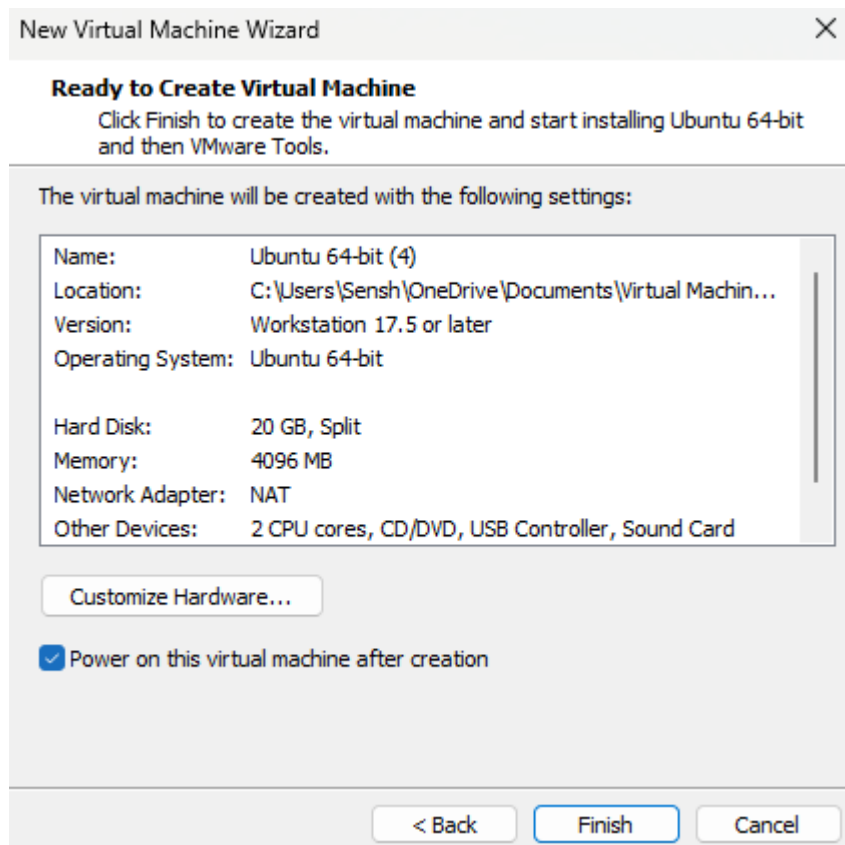
☐ Store virtual disk as a single file

☒ Split virtual disk into multiple files

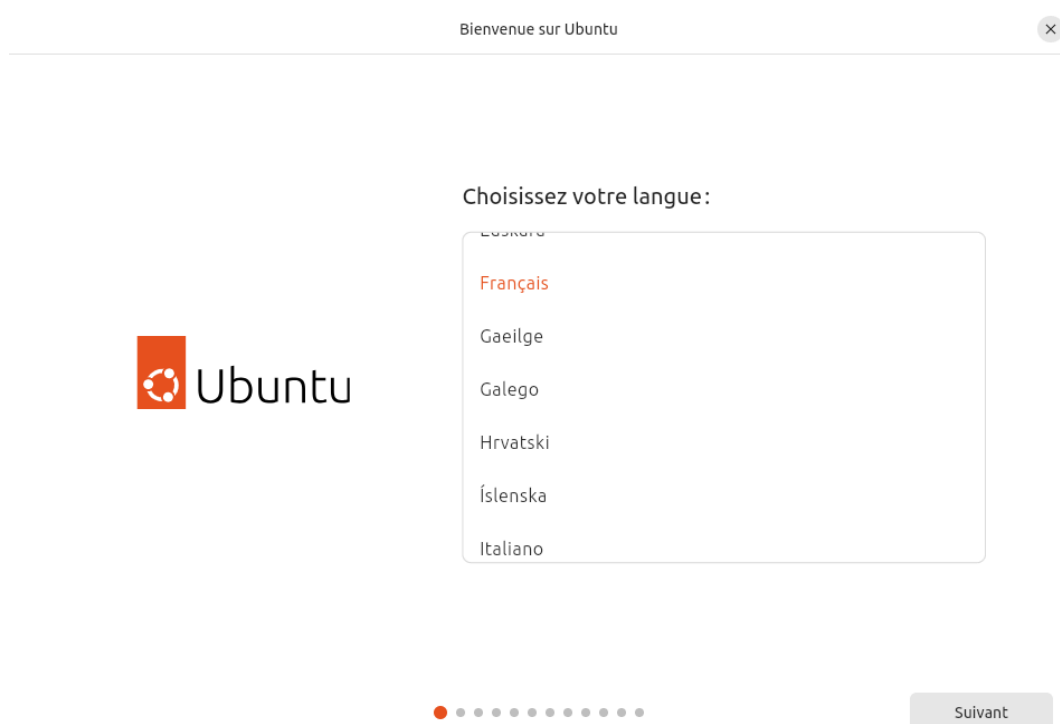
Splitting the disk makes it easier to move the virtual machine to another computer but may reduce performance with very large disks.

Help < Back Next > Cancel

7.6) Cliquez sur Finish



7.8) Patienter, puis cette page apparaîtra normalement :



7.9)



Accessibilité dans Ubuntu

Personnalisez Ubuntu selon vos besoins avant de l'installer. Vous pouvez faire des changements plus tard dans les paramètres système.



- | | |
|--|---|
|  Vision | > |
|  Audition | > |
|  Saisie | > |
|  Pointage et clic de souris | > |
|  Zoom | > |

Retour



Suivant

7.10)



Sélectionnez la disposition de votre clavier

Détecter



- Filipino
- Finnois
- Français**
- Français (Canada)
- Francais (Guinée)

Sélectionnez votre variante du clavier:

Français - Français (... ▾

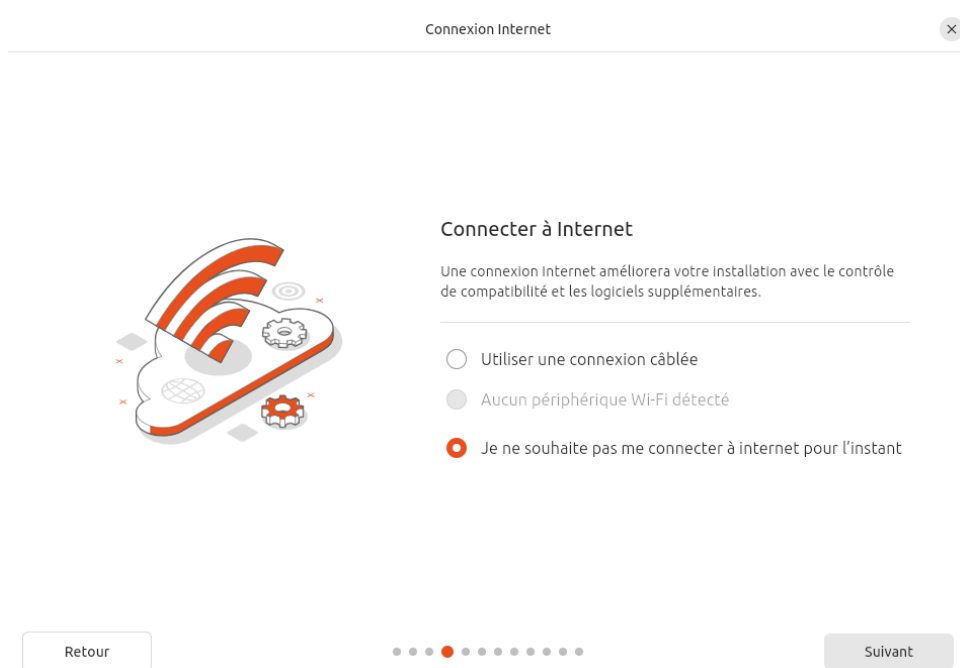
Saisissez du texte ici pour tester votre clavier

[Retour](#)

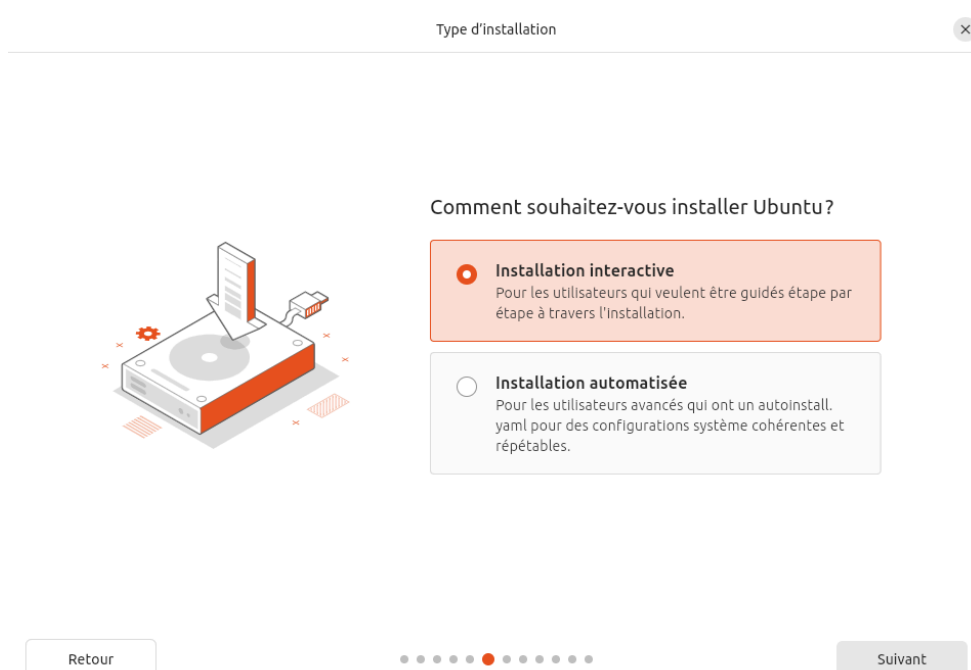


Suivant

7.11)



7.12)



7.13)



Quelles applications souhaitez-vous installer pour commencer ?

- ☒ **Installation par défaut**
Seulement les essentiels, navigateur Internet et utilitaires de base.
-
- ☐ **Installation complète**
Une sélection d'outils de bureau, utilitaires, navigateur Internet et jeux adaptée au hors ligne.

Retour



Suivant

7.14)



Installer les logiciels propriétaires recommandés?

Ubuntu est fourni sans logiciel propriétaire par défaut. L'installation de logiciels supplémentaires peut améliorer la performance de votre ordinateur.

- ☐ Installer des logiciels tiers pour le support du matériel graphique et Wi-Fi
Cela inclut mais ne se limite pas aux pilotes NVIDIA et assimilés
- ☐ Télécharger et installer la prise en charge de formats de multimédias supplémentaires
Cela inclut mais ne se limite pas aux MP3, MP4, MOV et assimilés

Retour



Suivant

7.15)

[illegible]

7.16)

Créer votre compte



Votre nom

User

✓

Le nom de votre ordinateur

user-VMware-Virtual-Platform

✓

Votre nom d'utilisateur

user

✓

Mot de passe

•••••

👁

Montrer

Mot de passe trop faible

Confirmez le mot de passe

•••••

✓

☐

Demander mon mot de passe pour ouvrir une s...

☐

Utiliser Active Directory

Retour

• • • • • • • • • •

•

• • • • •

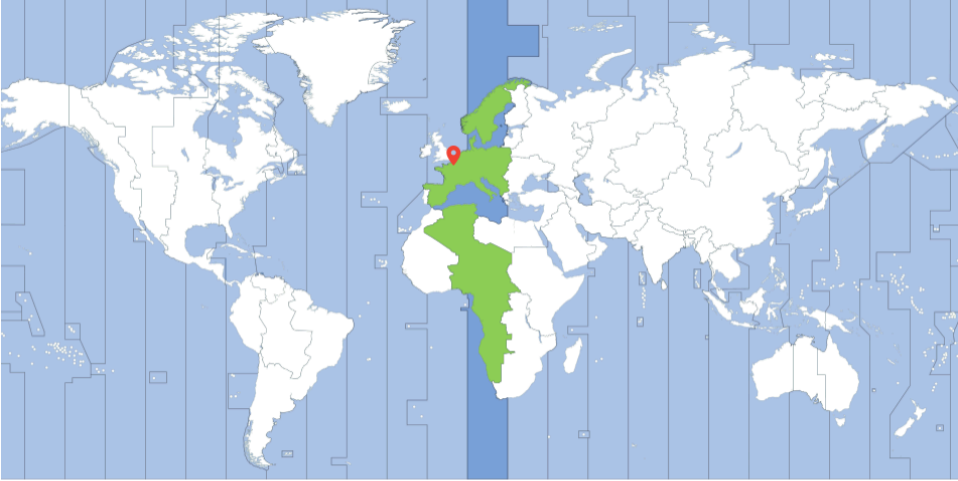
Suivant

7.17)

Sélectionnez votre fuseau horaire

Emplacement
Paris (Île-de-France, France)

Fuseau horaire
Europe/Paris



Retour

Suivant

7.18) Cliquez sur Installer

Prêt à installer



Vérifiez vos choix

Général

Configuration du disque

Disque d'installation

Applications

Sécurité & plus

Chiffrement du disque

Logiciel propriétaire

Partitions

la partition **sda1** créée

la partition **sda2** formatée en **ext4** utilisée pour /

Effacer le disque et installer Ubuntu

VMware Virtual S VMware, **sda**

Installation par défaut

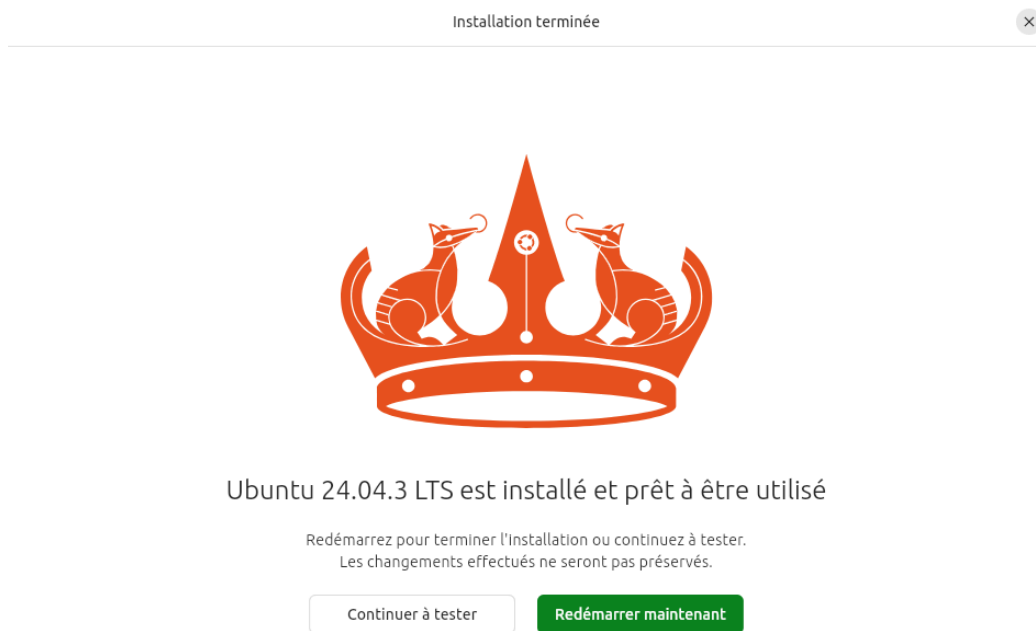
Aucune

None

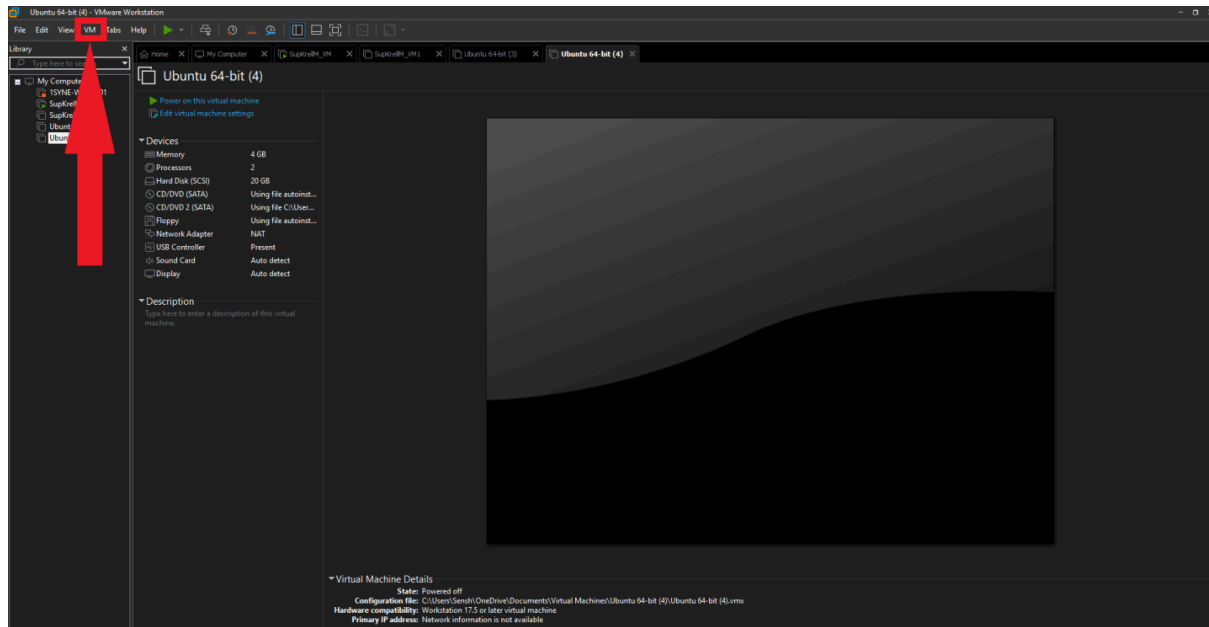
Retour

Installer

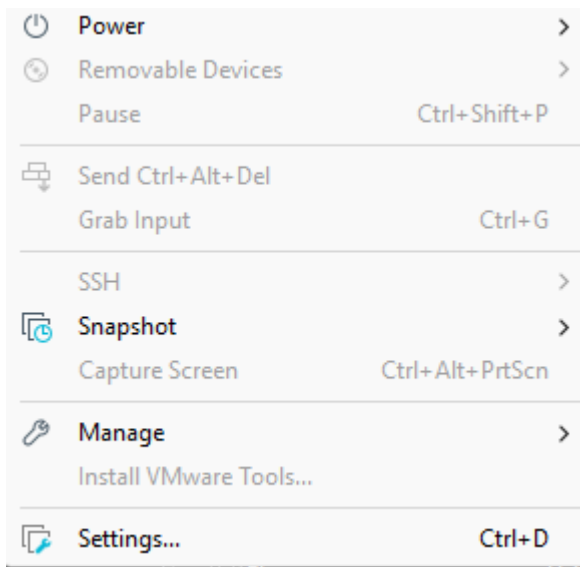
7.19) Redémarrer maintenant



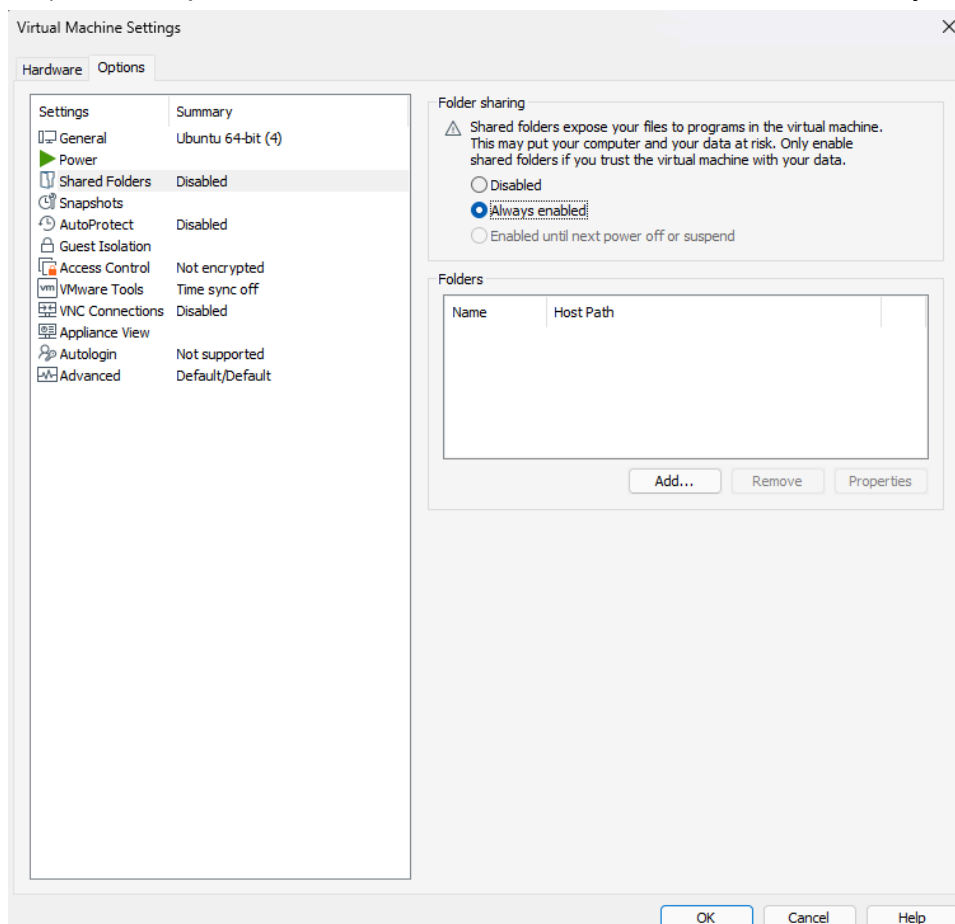
- 8) Dès qu'il est redémarré, faites POWER OFF
- 9) Ici, Cliquez sur "VM" et suivez les instructions :



9.1) Faites “Setting”



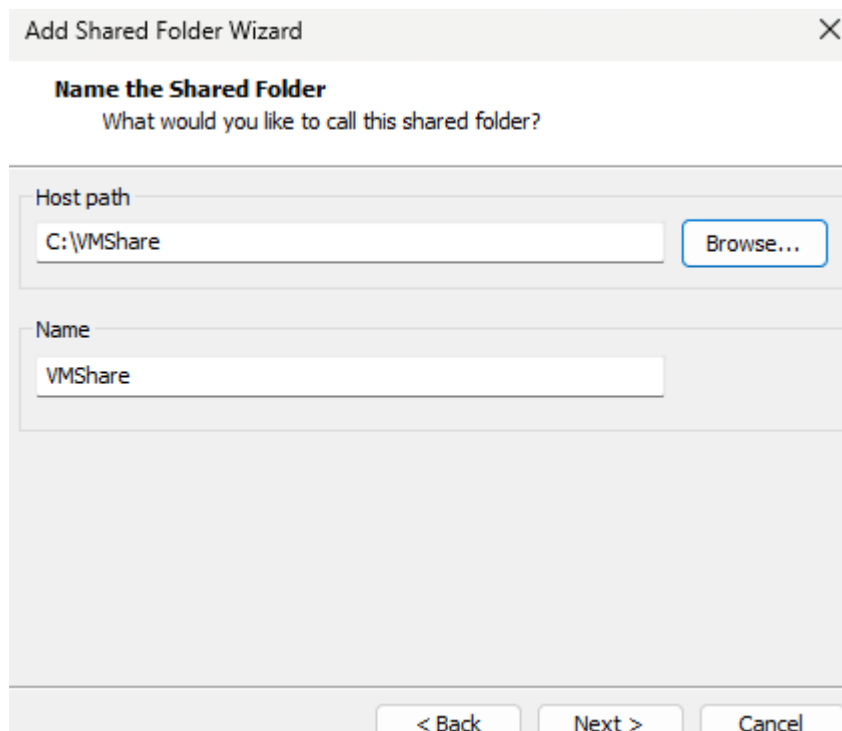
9.2) Dans “Options”, Allez dans Shared Folders et cochez “Always enabled”



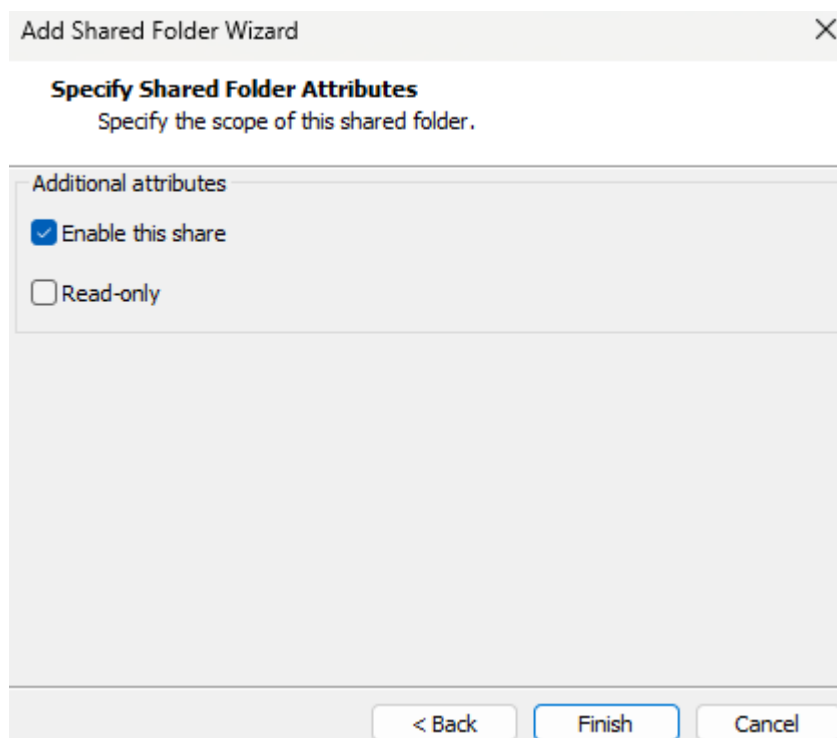
9.3) Cliquez sur “Add”



9.4) Mettez le chemin du dossier VMShare qui se trouve dans C:\ et nommez le VMShare et Cliquez “Next”



9.5)



9.6) Vous pouvez maintenant faire “Finish” puis “Ok” et lancer votre VM

10) Dans Linux, lancez le terminal et suivez les instructions :

10.1) Dans le terminal écrivez les commande suivante :

sudo apt install

sudo apt install python3-tk

```
user@user-VMware-Virtual-Platform:~$ sudo apt install
[sudo] Mot de passe de user :
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
0 mis à jour, 0 nouvellement installés, 0 à enlever et 102 non mis à jour.
user@user-VMware-Virtual-Platform:~$ sudo apt install python3-tk
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets supplémentaires suivants seront installés :
  blt libtk8.6 tk8.6-blt2.5
Paquets suggérés :
  blt-demo tk8.6 tix python3-tk-dbg
Les NOUVEAUX paquets suivants seront installés :
  blt libtk8.6 python3-tk tk8.6-blt2.5
0 mis à jour, 4 nouvellement installés, 0 à enlever et 102 non mis à jour.
Il est nécessaire de prendre 1 516 ko dans les archives.
Après cette opération, 4 929 ko d'espace disque supplémentaires seront utilisés.
```

10.2) Ensuite pour plus de confort, nous utiliserons **nano ~/.bashrc** pour éviter de réécrire la commande à chaque fois que l'on lance nos **.py**

10.3) Tout en bas du **nano ~/.bashrc**, écrivez cette commande:

```
rapport(){  
  
    cd /mnt/hgfs/VMShare || return  
  
    python3 "/mnt/hgfs/VMShare/rapport_supkrellm.py"  
  
}
```

```
rapport(){  
    cd /mnt/hgfs/VMShare || return  
    python3 "/mnt/hgfs/VMShare/rapport_supkrellm.py"  
}
```

10.4) Ensuite faites : **“CTRL + O”** puis **“Entrer”** puis **“CTRL + X”**

10.5) Toujours dans le Terminal faites :

sudo apt install

sudo apt install -y open-vm-tools open-vm-tools-desktop

```
user@user-VMware-Virtual-Platform:~$ sudo apt install  
Lecture des listes de paquets... Fait  
Construction de l'arbre des dépendances... Fait  
Lecture des informations d'état... Fait  
0 mis à jour, 0 nouvellement installés, 0 à enlever et 102 non mis à jour.  
user@user-VMware-Virtual-Platform:~$ sudo apt install -y open-vm-tools open-vm-tools-desktop  
Lecture des listes de paquets... Fait
```

10.6) puis redémarrez la VM avec : **sudo reboot**

10.7) Au redémarrage, lancez Terminal et exécutez (pour monter le shared folder):

Il faut impérativement faire ces commandes à chaque fois que l'on ouvre Terminal

```
sudo mkdir -p /mnt/hgfs
```

```
sudo vmhgfs-fuse .host:/ /mnt/hgfs -o allow_other
```

```
ls /mnt/hgfs
```

```
user@user-VMware-Virtual-Platform:~$ sudo mkdir -p /mnt/hgfs
user@user-VMware-Virtual-Platform:~$ sudo vmhgfs-fuse .host:/ /mnt/hgfs -o allow_other
user@user-VMware-Virtual-Platform:~$ ls /mnt/hgfs
VMShare
```

10.8) Si VMShare apparaît, c'est bon, vous pouvez maintenant exécuter le programme une première fois.

10.9) Pour plus de sécurité, à chaque fois que nous allons lancer **“rapport”** nous allons faire avant la commande : **source ~/.bashrc**

```
user@user-VMware-Virtual-Platform:~$ source ~/.bashrc
user@user-VMware-Virtual-Platform:~$ rapport
Rapport -> /mnt/hgfs/VMShare/rapport_supkrellm.html
```

Nous pouvons voir que les fichiers .html ont bien été ajouté à notre dossier “VMShare”

10.10) Pour l'arrêter, faites : **“CTRL + C”**

```
^CL'arrêt a été demandé.
```

11) Ouvrir le dossier VMShare sur votre machine principal et lancer le fichier **“rapport_supkrellm.html”** (ou le nom que vous lui avez attribué avec --page)

Ce PC > Windows > VMShare >

Nom	Modifié le	Type	Taille
__pycache__	28/11/2025 10:12	Dossier de fichiers	
collecte_systeme.py	28/11/2025 10:09	Fichier source Pyt...	16 Ko
donnees_systeme.html	28/11/2025 16:29	Microsoft Edge H...	16 Ko
rapport.html	28/11/2025 10:12	Microsoft Edge H...	5 Ko
rapport_supkrellm.html	28/11/2025 16:29	Microsoft Edge H...	13 Ko
rapport_supkrellm.py	28/11/2025 14:08	Fichier source Pyt...	9 Ko
templates_rapport.py	28/11/2025 09:12	Fichier source Pyt...	26 Ko
utils.py	28/11/2025 09:09	Fichier source Pyt...	1 Ko

12) Cette page devrait s'ouvrir normalement dans votre navigateur :

Rapport système - user-VMware-Virtual-Platform

Vue d'ensemble Matériel Mémoire Disques Processus Réseau Services web Erreurs

Vue d'ensemble

Date de génération 2025-11-28 16:29:26	Noyau Linux version 6.14.0-36-generic (build@lcy02-amd64-067) (x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0, GNU ld (GNU Binutils for Ubuntu) 2.42) #36~24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Wed Oct 15 15:45:17 UTC 2	Uptime 05:05:34
--	---	---------------------------

Matériel – Alimentation

Températures	Alimentation						
<table> <tr> <th>Capteur</th><th>Température</th><th>État</th></tr> <tr> <td>Donnée introuvable</td><td>Aucun capteur détecté</td><td>Erreur</td></tr> </table>	Capteur	Température	État	Donnée introuvable	Aucun capteur détecté	Erreur	Aucune batterie détectée
Capteur	Température	État					
Donnée introuvable	Aucun capteur détecté	Erreur					

Mémoire

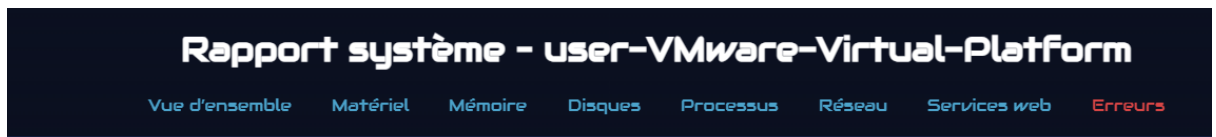
Totale 3.8 Go	Utilisée 1.5 Go (38.9%)	Libre + cache 2.3 Go
-------------------------	-----------------------------------	--------------------------------

Généré le 2025-11-28 [Rafraîchir les données](#)

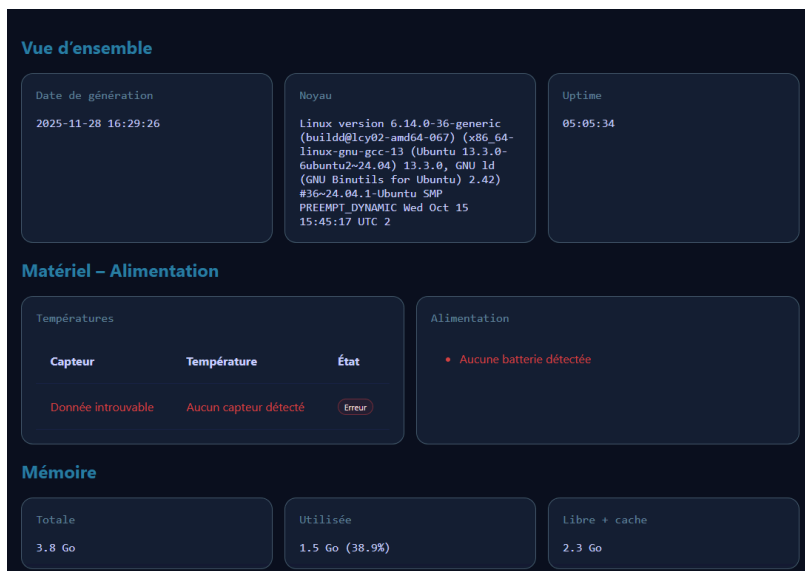
4.1. La page HTML

Sur cette page HTML vous pouvez voir le **header** tout en haut de la page. Au centre de cette page vous avez un **iframe** qui affiche donc les données du fichier “donnees_systeme.html” (ou le nom que vous lui avez attribué avec --donnees). Et pour finir, vous pouvez voir le **footer** en bas de la page qui possède la date de génération ainsi que le bouton “**Rafraîchir les données**”.

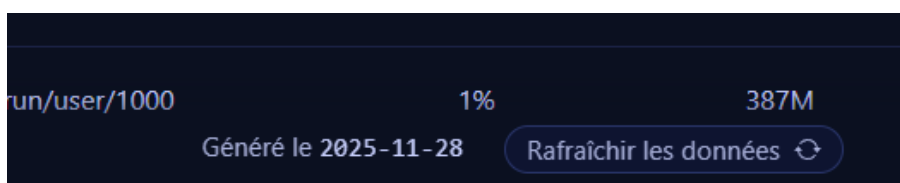
1) header :



2) iframe :



3) footer :



5. Méthodes de récupération des données

Dans cette partie, j'explique comment je récupère les informations du système, et pourquoi j'ai choisi ces méthodes-là. Tout ce que je décris ici correspond vraiment à ce qui se passe dans mon script Python.

5.1 Informations générales

1) **Nom d'hôte**

Je lis le fichier système :

/proc/sys/kernel/hostname

S'il est illisible, j'ajoute l'erreur et je mets "inconnu".

2) **Version du noyau**

Je lis le fichier :

/proc/version

Ce fichier donne la version exacte du noyau Linux.

3) **Uptime (durée de fonctionnement)**

Je lis :

/proc/uptime

Je prends la première valeur (en secondes), puis je la convertis en format lisible grâce à la fonction `format_duree()`.

4) **Date et heure**

J'utilise le module Python `datetime` pour générer les jetons `DATE` et `DATE_HEURE`.

Ces fichiers sont présents sur toutes les distributions Linux, donc c'est fiable.

5.2 Mémoire

1) **Source utilisée**

Je lis le fichier :

/proc/meminfo

2) **Valeurs récupérées**

a) `MemTotal`

- b) MemFree
- c) Buffers
- d) Cached
- e) SReclaimable
- f) Shmem

3) **Traitement**

- a) Je convertis tout en Go
- b) Je calcule :
 - b.a) mémoire totale
 - b.b) mémoire utilisée
 - b.c) mémoire libre + cache
 - b.d) pourcentage d'utilisation

4) **En cas d'erreur**

Je remplis avec "Donnée non disponible" et j'ajoute l'erreur dans la section correspondante.

5.3 Températures

1) **Je parcours les fichiers :**

/sys/class/thermal/thermal_zone*/temp

2) **Pour chaque capteur**

- a) je lis la température brute (en milli-degrés)
- b) je la divise par 1000
- c) j'affiche une ligne avec un badge OK

3) **En machine virtuelle**

Souvent il n'y a aucun capteur → j'affiche "Aucun capteur détecté".

5.4 Alimentation

1) **Je cherche les dossiers :**

/sys/class/power_supply/BAT*/

2) **Si je trouve une batterie**

- a) je lis status
- b) je lis capacity
- c) je construis une ligne comme "BAT0: Charging — 78%"

- 3) **Si je ne trouve rien**
J'affiche : "Aucune batterie détectée".

5.5 Disques

- 1) **Commande utilisée**
df -T -hP
- 2) **Traitement**
 - a) Je lance la commande avec subprocess.run()
 - b) J'analyse chaque ligne
 - c) Je récupère :
 - a. périphérique
 - b. type
 - c. utilisation
 - d. espace libre
 - e. point de montage
- 3) **En cas d'erreur**
J'affiche "Donnée introuvable".

5.6 Processus

- 1) **Commande utilisée**
ps aux --sort=-%cpu
- 2) **Traitement**
 - a) je garde les 10 premiers
 - b) je récupère :
 - a. utilisateur
 - b. PID
 - c. CPU%
 - d. RAM%
 - e. commande
 - c) la commande est tronquée à 120 caractères
- 3) **Si erreur**
Je mets une ligne "Donnée introuvable".

5.7 Interfaces réseau

- 1) **Adresses IP**
 - a) je lance : `ip -j addr`
 - b) je récupère IPv4 et IPv6
- 2) **Statistiques RX/TX**

Je lis : `/proc/net/dev`
- 3) **État des interfaces**

Je lis : `/sys/class/net/<iface>/operstate`
Et j'affiche l'état avec un badge (ok, warn, err).
- 4) **Fusion**

Toutes les infos sont regroupées en une ligne par interface.

5.8 Connexions réseau

- 1) **Commande utilisée**

`ss -tuln`
- 2) **Traitement**
 - a) j'analyse chaque ligne
 - b) je mets chaque connexion sous forme de `<li>`
- 3) **Si aucune connexion**

J'affiche "Aucune connexion".

5.9 Services web

- 1) **Détection**
 - a) je scanne les ports avec : `ss -ntlp`
 - b) je garde seulement le port 80 ou 443
 - c) je remplace * ou 0.0.0.0 par 127.0.0.1
- 2) **Éviter les doublons**

J'utilise un set (host, port).

3) **Connexion**

- a) port 80 → HTTPConnection
- b) port 443 → HTTPSConnection
- c) timeout = 2 secondes
- d) je fais une requête GET sur /

4) **Extraction des infos**

- a) titre de la page
- b) favicon = /favicon.ico
- c) serveur (header Server)
- d) protocole (HTTP/HTTPS)
- e) statut HTTP

5) **Gestion des erreurs**

Je capture tout et je mets des valeurs par défaut.

6) **Résultat**

Chaque service web détecté apparaît sous forme d'une ligne HTML.

6. Mode Graphique (TKinter)

6.1 Le mode "--gui"

Le projet possède un paramètre (--gui) qui permet de voir les données pas sur un page HTML mais sur un interface TKinter.

6.2 Comment le lancer

Pour lancer le mode "--gui" il faut :

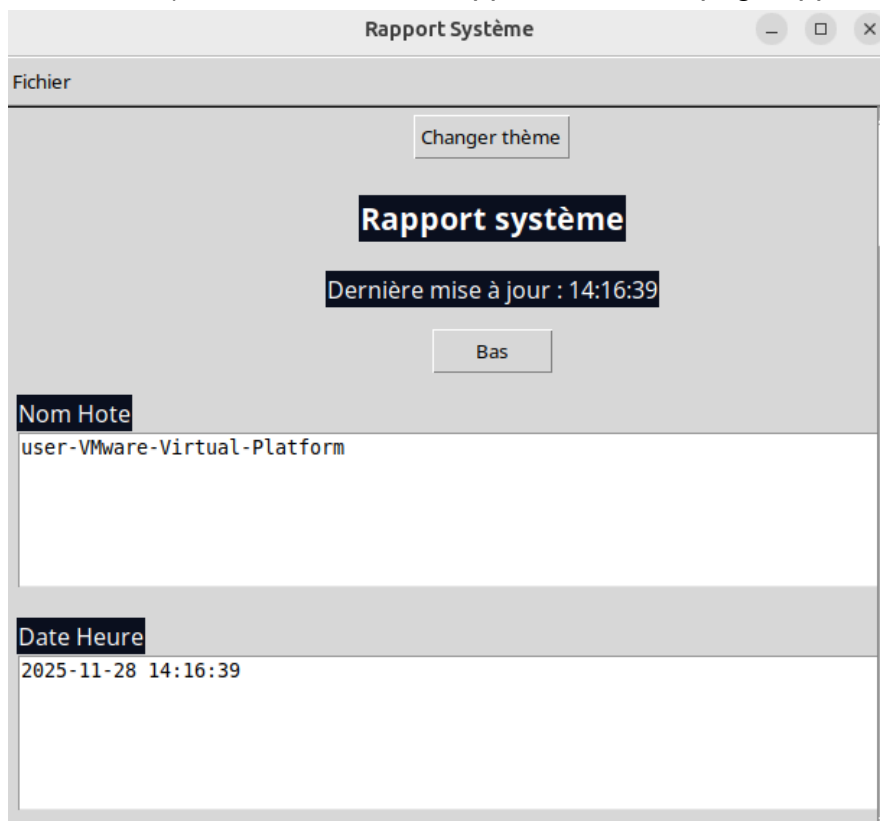
6.2.1) retourner dans nano ~/.bashrc et écrire --gui ici :

```
rapport(){  
    cd /mnt/hgfs/VMShare || return  
    python3 "/mnt/hgfs/VMShare/rapport_supkrellm.py" --gui  
}
```

6.2.2) Ensuite faites : **"CTRL + O"** puis **"Entrer"** puis **"CTRL + X"**.

6.2.3) Pensez à faire **source ~/.bashrc** avant de lancer le rapport.

6.2.4) Maintenant faites rapport et si cette page apparaît :



c'est que vous y êtes arrivé !

6.3 Contenus dans cette interface

6.3.1 L'interface possède plusieurs paramètres :

- 1) **Changer thème** : permet de switcher entre un thème sombre et un thème clair.
- 2) **Bas** : qui permet d'aller tout en bas de la page en un seul clique.
- 3) **Haut** : qui permet de remonter toute la page en un seul clique également.
- 4) **Exporter HTML** : qui se trouve dans "Fichier", permet d'afficher cette interface dans une page html (sans passer par le terminal et enlever le --gui).

6.3.2 Le contenu qu'on y retrouve :

- 1) Nom de L'Hôte
- 2) Date et Heure
- 3) Noyau
- 4) Durée de Fonctionnement
- 5) Mémoire Total / Mémoire Utilisé (%) / Mémoire Libre
- 6) Lignes Température
- 7) Éléments Alim
- 8) Lignes Disques
- 9) Lignes Processus
- 10) Lignes Interfaces
- 11) Éléments Connexions
- 12) Lignes Web
- 13) Erreurs

6.4 Comment il fonctionne

Dans cette fenêtre, je n'utilise aucune boucle qui bloque le programme (comme une boucle "**while True**" dans l'interface).

À la place, j'utilise la fonction **".after()"** de Tkinter.

.after() permet d'actualiser l'écran régulièrement sans faire freezer la fenêtre.

Par exemple, toutes les 2 secondes, je rappelle une fonction qui met à jour les données.

Les informations affichées dans la fenêtre viennent du même code que celui utilisé dans le rapport HTML.

Elles sont donc toujours à jour et cohérentes.

La fenêtre reste fluide car Tkinter continue de gérer ses propres événements (affichage, clics, etc.), pendant que mon code actualise les valeurs en arrière-plan.

7. Les Paramètres du script

7.1 Mini Introduction

Le projet contient au total 11 paramètres différents (dont --gui).

7.2 Les paramètres et leur fonctionnalités

7.2.1 Les paramètres

- 1) --page
- 2) --donnees
- 3) --intervalle
- 4) --dossier
- 5) --gui
- 6) --sans-memoire
- 7) --sans-disques
- 8) --sans-processus
- 9) --sans-reseau
- 10) --sans-web
- 11) --sans-materiel

7.2.2 Les fonctionnalités

--**page** permet de changer le nom du fichier interface (rapport_supkrellm) en .html à sa sortie.

--**donnees** permet de changer le nom du fichier des données (donnees_systeme.) en .html à sa sortie.

--**intervalle** permet de changer le l'intervalle de temps qu'il y a à chaque fois que la VM donne ses informations (il est conseillé de ne pas changer cette valeur pour éviter les crashes (valeur à 2 par défaut)).

--**dossier** permet de choisir le dossier dans lequel vont se créer les fichiers html (Voir la partie 7.2.3 pour plus d'informations).

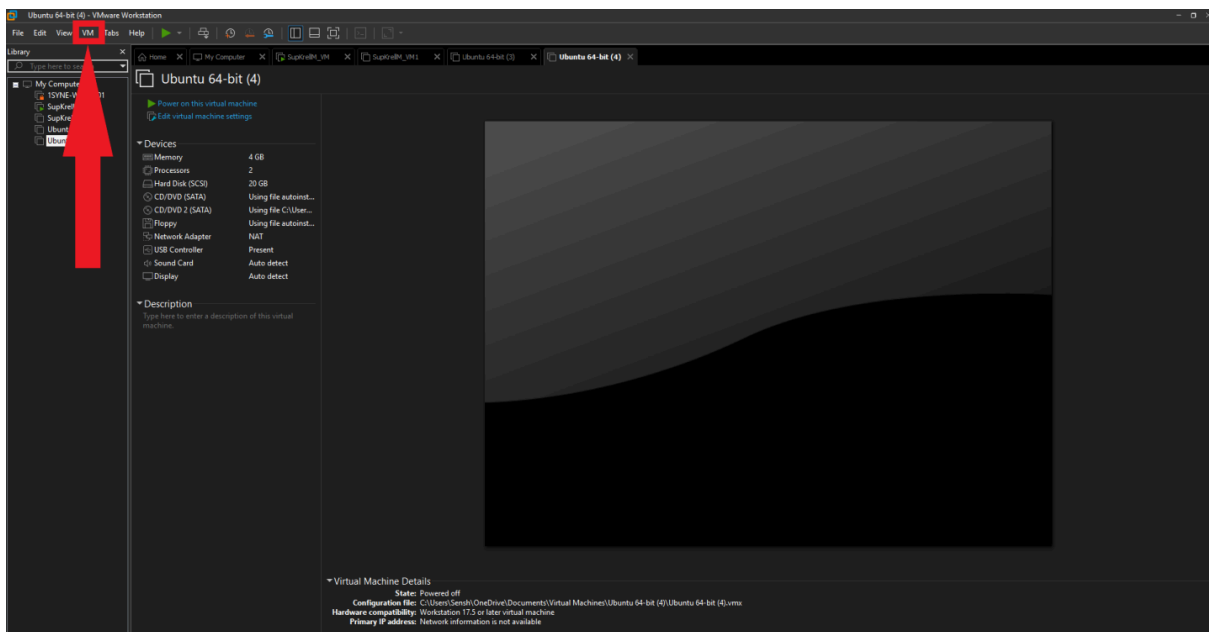
--gui qui permet donc de lancer l'interface TKinter (Voir la partie 6.2 pour plus d'informations).

--sans-memoire / disques / processus / reseau / web / materiel permet de retirer une ou plusieurs métriques souhaité dans le rapport.

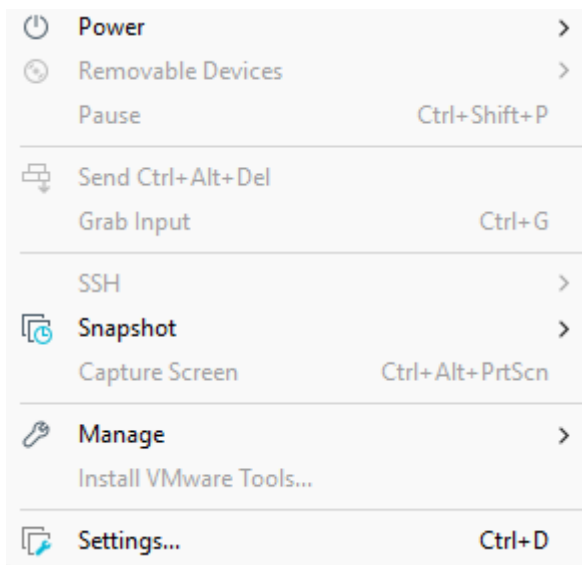
7.2.3 --dossier

Pour que le --dossier fonctionne il faut :

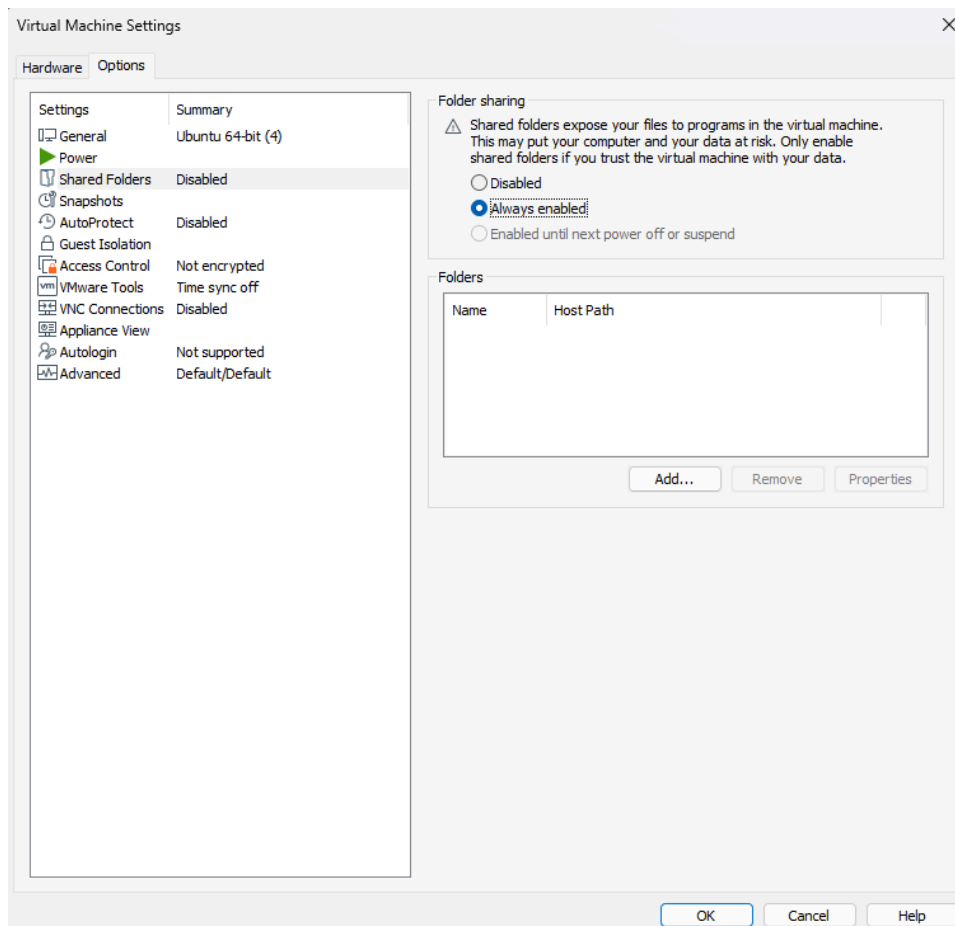
- 1) Allez dans VM en haut à gauche de votre écran



- 2) Faites "Setting"



3) Dans “Options”, Allez dans Shared Folders



4) Cliquez sur “Add”



- 5) Mettez le chemin du dossier de votre choix comme C:\Users\User\Bureau et donnez lui un nom simple pour le retenir (ici c'est Bureau)

The screenshot shows the 'Add Shared Folder Wizard' dialog box. The title bar says 'Add Shared Folder Wizard' with a close button. The main heading is 'Name the Shared Folder' with the instruction 'What would you like to call this shared folder?'. There are two input fields: 'Host path' containing 'C:\Users\Sensh\OneDrive\Bureau' and a 'Browse...' button, and 'Name' containing 'Bureau'. At the bottom are '< Back', 'Next >', and 'Cancel' buttons.

- 6) Vous pouvez maintenant faire "Finish" puis "Ok" et lancer votre VM

The screenshot shows the 'Add Shared Folder Wizard' dialog box at the 'Specify Shared Folder Attributes' step. The title bar says 'Add Shared Folder Wizard' with a close button. The main heading is 'Specify Shared Folder Attributes' with the instruction 'Specify the scope of this shared folder.'. Under 'Additional attributes', there are two checkboxes: 'Enable this share' (checked) and 'Read-only' (unchecked). At the bottom are '< Back', 'Finish', and 'Cancel' buttons.

- 7) Dans Linux, lancer le Terminal
- 8) Puis écrire :

```
rapport(){  
  
    cd /mnt/hgfs/VMSHare || return  
  
    python3 "/mnt/hgfs/VMSHare/rapport_supkrellm.py" --dossier  
/mnt/hgfs/Bureau  
  
}
```

(de cette manière)

```
rapport(){  
    cd /mnt/hgfs/VMSHare || return  
    python3 "/mnt/hgfs/VMSHare/rapport_supkrellm.py" --dossier /mnt/hgfs/Bureau  
}
```

- 9) Faites : **"CTRL + O"** puis **"Entrée"** puis **"CTRL + X"**
- 10) Ensuite il faut penser à écrire : **source ~/.bashrc** et vous pouvez lancer **"rapport"**
- 11) De cette manière, vous retrouverez les fichiers **.html** sur votre Bureau (de votre machine principal)

8. Gestion des Erreurs

- 1) Tout au long de l'exécution du programme, je fais en sorte qu'il n'y ait jamais de crash. Même si une commande ne répond pas ou qu'un fichier système manque, le script continue normalement.
- 2) Toutes les erreurs que je rencontre pendant la récupération des données sont stockées dans une liste, puis affichées dans la section "Erreurs" du rapport HTML. Ça permet de comprendre ce qui ne marche pas, sans casser la page.
- 3) Quand une donnée est introuvable, je la remplace toujours par une indication claire. Par exemple, ça peut afficher "Donnée non disponible" ou "Aucune batterie détectée".
- 4) J'utilise aussi des couleurs pour rendre immédiatement visibles les problèmes :
 - a) rouge pour les erreurs "err" (par exemple, si un capteur température ne répond pas)
 - b) badge "OK" (vert) quand la donnée est correcte
 - c) badge "warn" (orange) si l'info est partielle ou incertaine
- 5)

Grâce à ces choix, le rapport garde un affichage propre et compréhensible, même si le système est limité (comme dans une machine virtuelle). On a toujours une information utile à l'écran, jamais un écran vide ou un plantage.

9. Limitations Connues

- 1) Sur une machine virtuelle, certains composants matériels n'existent pas ou ne sont pas accessibles. Par exemple, il n'y a souvent aucun capteur de température détectable dans **/sys/class/thermal**. Dans ce cas, j'affiche simplement un message clair au lieu de planter.
- 2) La gestion de la batterie ne fonctionne pas sur les PC fixes ni en VM, car il n'y a pas de batterie physique dans ces systèmes. Le rapport affiche donc **"Aucune batterie détectée"**, ce qui est normal.
- 3) Les services web locaux sur les ports 80 et 443 ne sont pas toujours présents. Du coup, la section Services Web peut être vide même si le programme fonctionne très bien. C'est plus une particularité de la machine qu'une particularité du programme.
- 4) Le script dépend de certaines commandes système comme **ip**, **ss**, **df** ou **ps**. Si l'une d'elles est manquante, la récupération liée à cette commande échoue. Mais j'ai prévu des messages d'erreurs pour le signaler proprement dans le rapport.
- 5) Certaines informations peuvent changer en fonction des droits de l'utilisateur. Si le script tourne avec des permissions limitées, il peut manquer quelques données, mais le rapport s'affiche quand même.

10. Conclusion

10.1

Le programme respecte toutes les consignes du projet. Il récupère et affiche les infos système demandées sans utiliser le module os ni aucune librairie extérieure. S'il y a une erreur ou une info impossible à récupérer, le programme ne plante pas et l'indique clairement dans le rapport.

10.2

Le rapport HTML est clair, lisible et propre à consulter, avec une mise en page organisée pour comprendre vite l'état de la machine. En plus de ça, il y a aussi une interface graphique Tkinter qui permet de voir les infos en temps réel, ce qui apporte un vrai confort d'utilisation.

10.3

Le projet est aussi améliorable facilement, car la façon dont il est organisé permet de rajouter de nouvelles métriques plus tard si besoin.

Fin du fichier